

Task2View

Multi-Agent Generation of Stakeholder-Oriented Architecture Views



Why this problem matters

Architecture views are useful only when they answer the right stakeholder questions.

Current limitation

Generic documentation often produces fixed sections or diagrams, independent of the concrete task.

Real need

Different stakeholders require different information, concerns, and levels of abstraction.

Research gap

The system must decide which views are needed and whether they are grounded in the repository.

Examples of stakeholder-specific needs

Developer

component responsibilities and dependencies

Tester

runtime interactions and failure paths

Maintainer

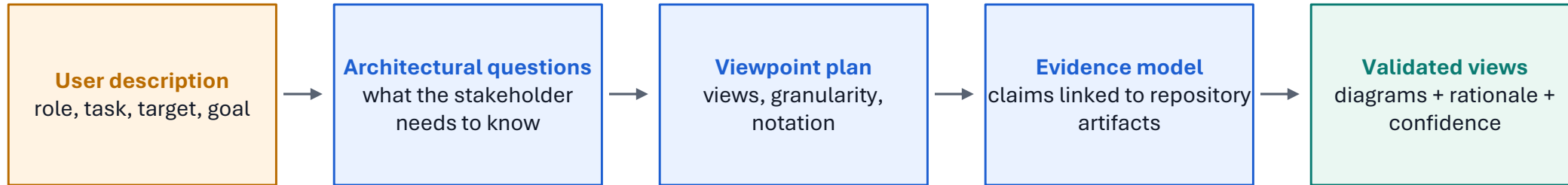
change impact and dependency views

Security analyst

trust boundaries and sensitive data flows

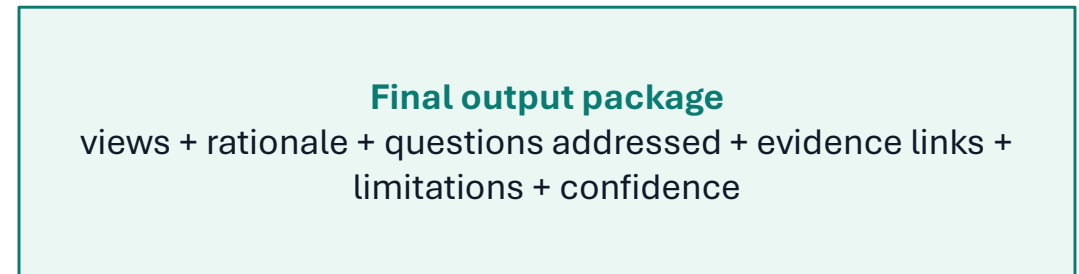
Core process

The system transforms a user need into validated, evidence-grounded views.

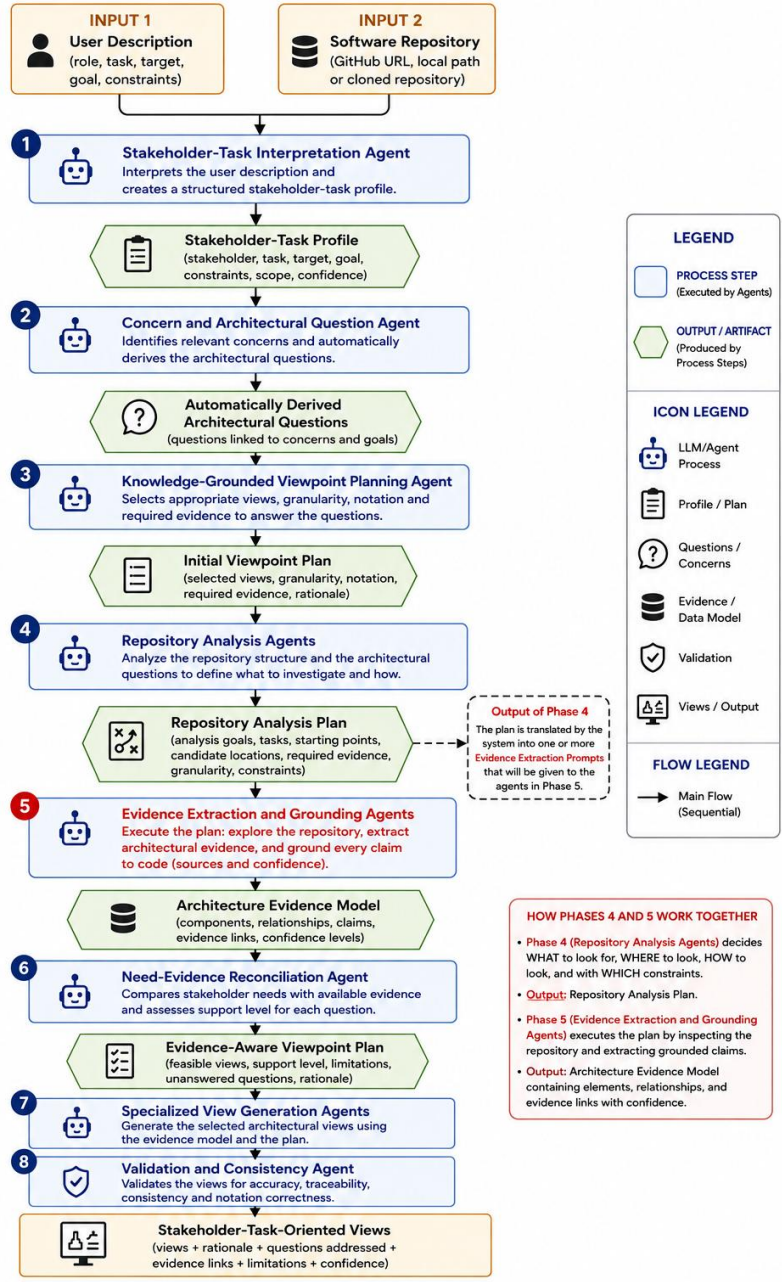


Key principle

Do not ask the user to know which diagram is needed. Let the agents infer the architectural questions first, then select the views.



Overall multi-agent architecture



What the figure clarifies

Blue boxes are agent-executed process steps; green boxes are intermediate artifacts.

Critical bridge

Phase 4 outputs a Repository Analysis Plan, which becomes input for Phase 5 extraction prompts.

Why it matters

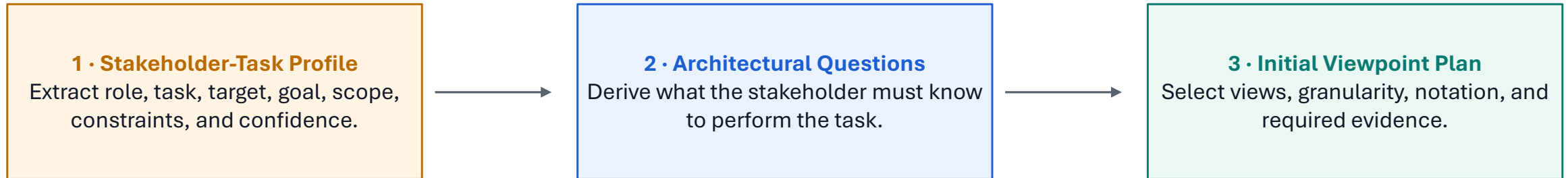
The system separates planning the investigation from extracting grounded architectural evidence.

Inputs and Outputs by Phase

Phase	Agent / Component	Inputs	Outputs
1	Stakeholder-Task Interpretation Agent	User description	Stakeholder-Task Profile
2	Concern and Architectural Question Agent	Stakeholder-Task Profile	Architectural Concerns and Questions
3	Knowledge-Grounded Viewpoint Planning Agent	Profile, questions, architecture knowledge base	Initial Viewpoint Plan
4	Repository Analysis Agents	Repository, questions, Initial Viewpoint Plan	Repository Analysis Plan and extraction prompts
5	Evidence Extraction and Grounding Agents	Repository, Repository Analysis Plan, prompts	Grounded claims and Architecture Evidence Model
6	Need-Evidence Reconciliation Agent	Initial Viewpoint Plan, questions, evidence model	Evidence-Aware Viewpoint Plan
7	Specialized View Generation Agents	Evidence-aware plan and evidence model	Candidate Architecture Views
8	Validation and Consistency Agent	Candidate views, evidence model, questions, profile	Validated Stakeholder-Task-Oriented Views

Phases 1–3: from user need to viewpoint plan

The user gives a natural-language need; agents derive questions and select candidate views.



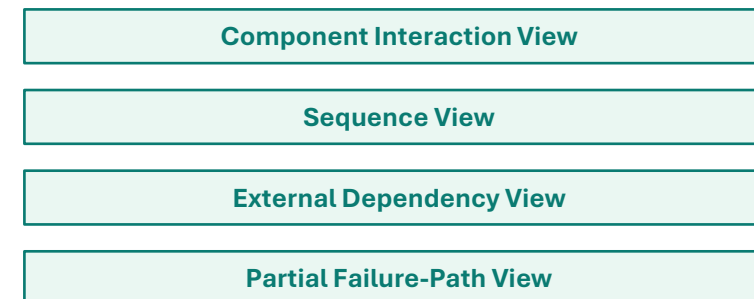
Example

“I am an integration tester. I need to create tests for the payment functionality and understand which components are involved, in which order they interact, which external services are used, and which failure paths should be tested.”

Derived questions

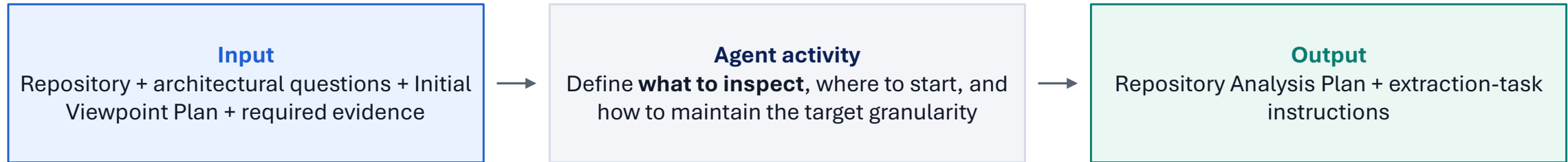
- Which components participate?
- In which order do they interact?
- Which external services are involved?
- Which failure paths are implemented?

Selected views



Phase 4: Repository Analysis Agents

This phase plans the investigation; it does not yet extract the final architectural evidence.



Repository Analysis Plan contains



Example instruction

“Start from controllers, services, repositories, clients and integration tests. Recover the component-level payment workflow. Ground every relationship.”

Conceptual role: Phase 4 defines what, where, and how to investigate.

Phase 5: Evidence Extraction and Grounding Agents

This phase executes the plan and produces grounded architectural claims.

Evidence Extraction Prompt

Recover the component-level payment workflow. For each interaction, report source, target, type, order, file, symbol, excerpt, support type, and confidence.

LLM agents inspect
files, references, tests,
configuration, API routes

Grounded claims
elements + relationships +
evidence links

Atomic claim example

```
{  
  "claim": "PaymentService invokes StripeClient",  
  "relationship": "invokes",  
  "evidence": "PaymentService.processPayment",  
  "support_type": "directly_observed",  
  "confidence": 0.97  
}
```

Phase 4 vs Phase 5

Phase 4	plans the investigation
Phase 5	executes it and grounds claims
Phase 4 output	Repository Analysis Plan
Phase 5 output	Architecture Evidence Model

Architecture Evidence Model

A shared intermediate representation separates repository interpretation from view rendering.

Elements

components, services, packages, external systems, data stores

Relationships

calls, dependencies, data flows, invocations, deployment links

Evidence metadata

file, symbol, excerpt, support type, confidence

Why this matters

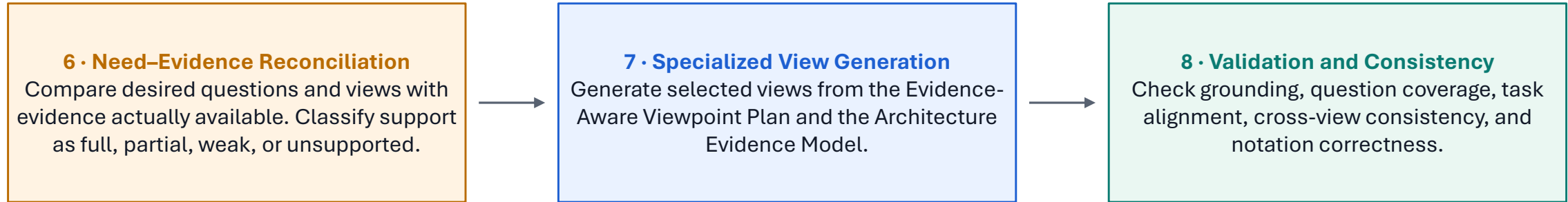
- prevents each view agent from independently reinterpreting the repository
- enables traceability from views to code
- supports consistency across multiple views
- allows confidence and limitations to be made explicit

Example abstraction

```
Payment API → Payment Service
Payment Service → Payment Data Store
Payment Service → External Payment Provider
```

Phases 6–8: evidence-aware view generation

The system generates only views that are sufficiently supported by repository evidence.



The validation gate prevents unsupported views from being delivered as facts.



Evaluation strategy

Evaluate whether we generated the right views, whether they help the task, and whether they are faithful to the repository.

Stakeholder-Task Adequacy

Did the system correctly interpret the stakeholder and task, derive relevant questions, and select appropriate views and granularity?

Task Utility

Do the generated views help users complete their task with better correctness, confidence, and lower effort?

Repository Faithfulness

Are view elements and relationships supported by repository artifacts, with traceability and consistency?

Possible measures

view-selection relevance

task completion time

task answer correctness

unsupported claim rate

traceability coverage

human correction effort

Research questions

A concise set of RQs aligns with the three evaluation dimensions.

RQ1 · Stakeholder-task interpretation and view planning

To what extent can the approach derive appropriate architectural questions and select views relevant to a stakeholder's task?

RQ2 · Task support

To what extent do the generated architecture views support stakeholders in performing their intended tasks?

RQ3 · Repository faithfulness

To what extent are the generated architecture views faithful to the analyzed software repository?

Optional RQ4 · Efficiency and correction effort

Measure execution time, token cost, cost per view, number of corrections, and human validation time.

Expected contributions and meeting alignment

The discussion should converge on scope, evaluation, and prototype responsibilities.

Expected contributions

1. stakeholder-task-driven formulation of view generation
2. architectural-question derivation and viewpoint planning
3. Repository Analysis Plan as a bridge between planning and extraction
4. LLM-based evidence extraction and grounding
5. Architecture Evidence Model for traceable view generation
6. need-evidence reconciliation and validation framework

Decision 1

Which stakeholder-task scenarios should we prioritize for the first prototype?

Decision 2

Which view types and notations should be supported in the initial study?

Decision 3

Which baselines and metrics are realistic for the meeting paper deadline?

Positioning statement

The approach does not ask an LLM to simply generate a diagram. It first determines who needs the information, what must be answered, which view is appropriate, what evidence is available, and only then generates a validated view.